

13 | MVCC协议：MySQL 在修改数据的时候，还能不能读到这条数据？

2023-7-14 邓明



你好，我是大明。今天我们来学习 MySQL 面试中非常重要的一个内容——MVCC 协议。

MVCC (Multi-Version Concurrency Control) 中文叫做多版本并发控制协议，是 MySQL InnoDB 引擎用于控制数据并发访问的协议。它在面试中属于必面题，而且从 MVCC 出发能够将话题引申到事务、隔离级别两个重头戏上，所以掌握 MVCC 能让你进可攻退可守。

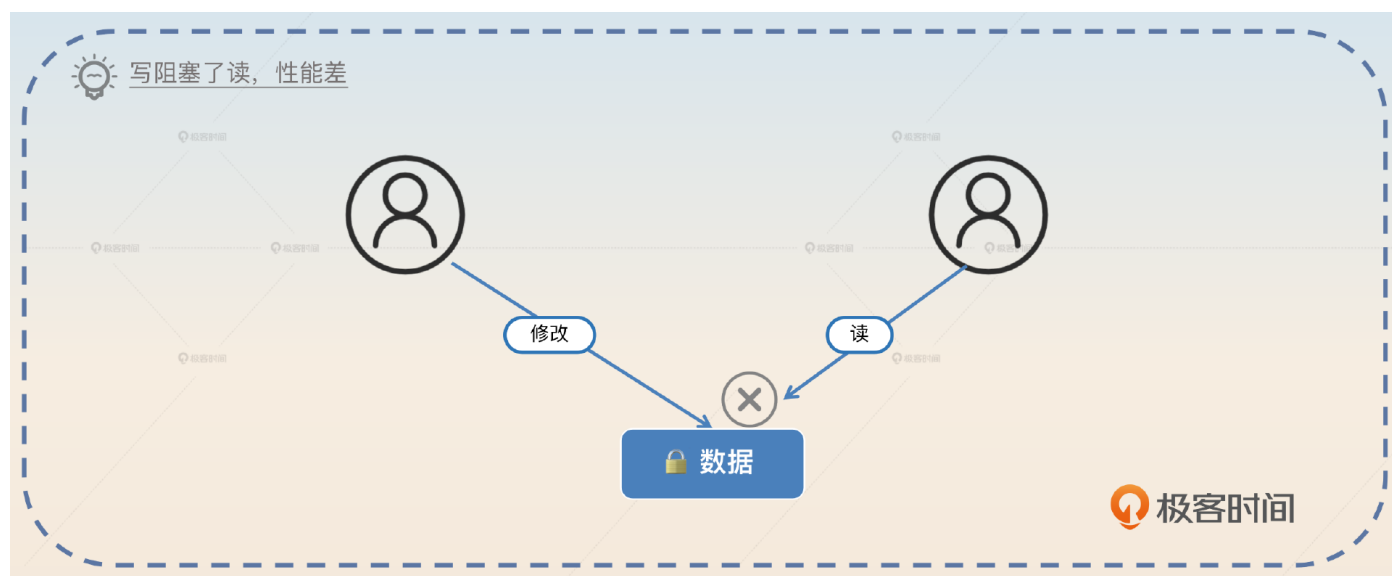
那么今天我就带你从 MVCC 的基本原理开始讲起，教你怎么在 MVCC 的面试中进退自如，秀出实力。在开始之前，我们先思考一个问题，为什么 InnoDB 会需要 MVCC？

为什么需要 MVCC？

你在前面已经学过了锁，知道锁本身就是用于并发控制的，那么为什么 InnoDB 还需要引入 MVCC，读写都加锁不就可以控制住并发吗？

锁确实可以，但是性能太差。如果是纯粹的锁，那么写和写、读和写、读和读之间都是互斥的。如果是读写锁，那么写和写、读和写之间依旧是互斥的。

数据库和一般的应用有一个很大的区别，就是**数据库即便是读，也不能被写阻塞住**。试想一下，如果一个线程准备执行 UPDATE 一行数据，如果这时候阻塞住了所有的 SELECT 语句，那么这个性能你能接受吗？



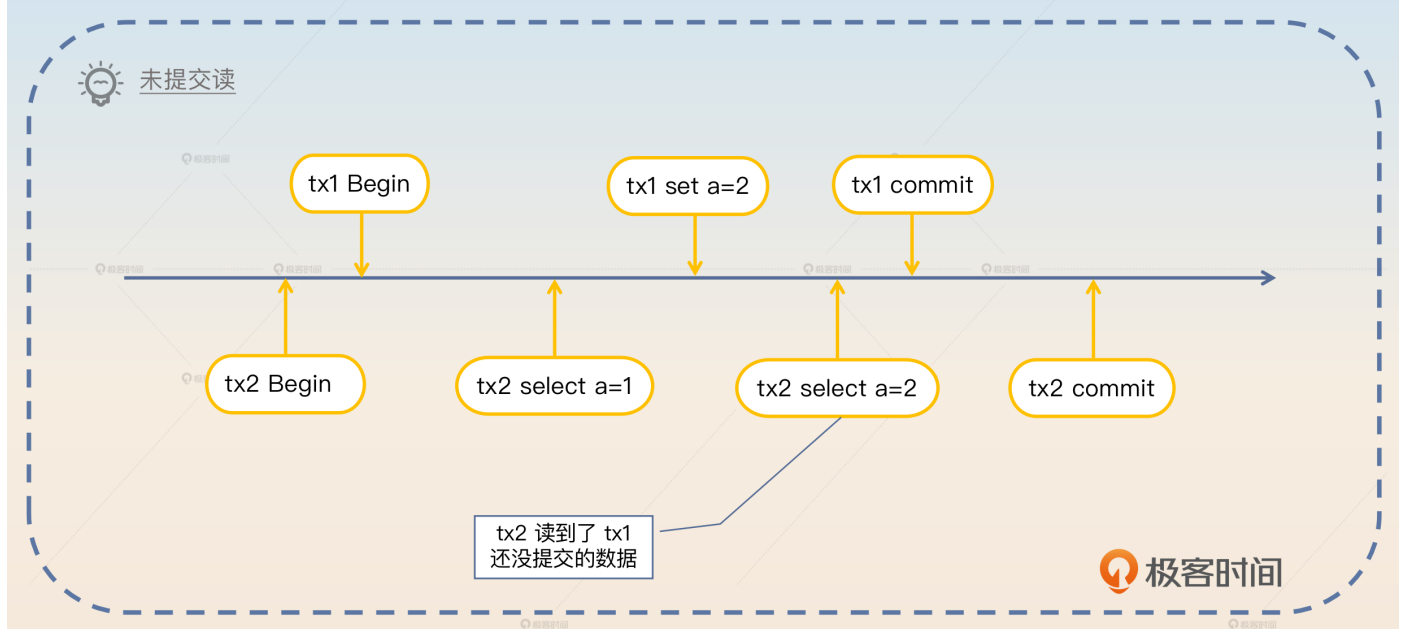
显然接受不了，所以数据库要有一种机制，避免读写阻塞。在理解了为什么 MVCC 必不可少之后，现在你需要进一步了解一个和 MVCC 紧密关联的概念：**隔离级别**。

隔离级别

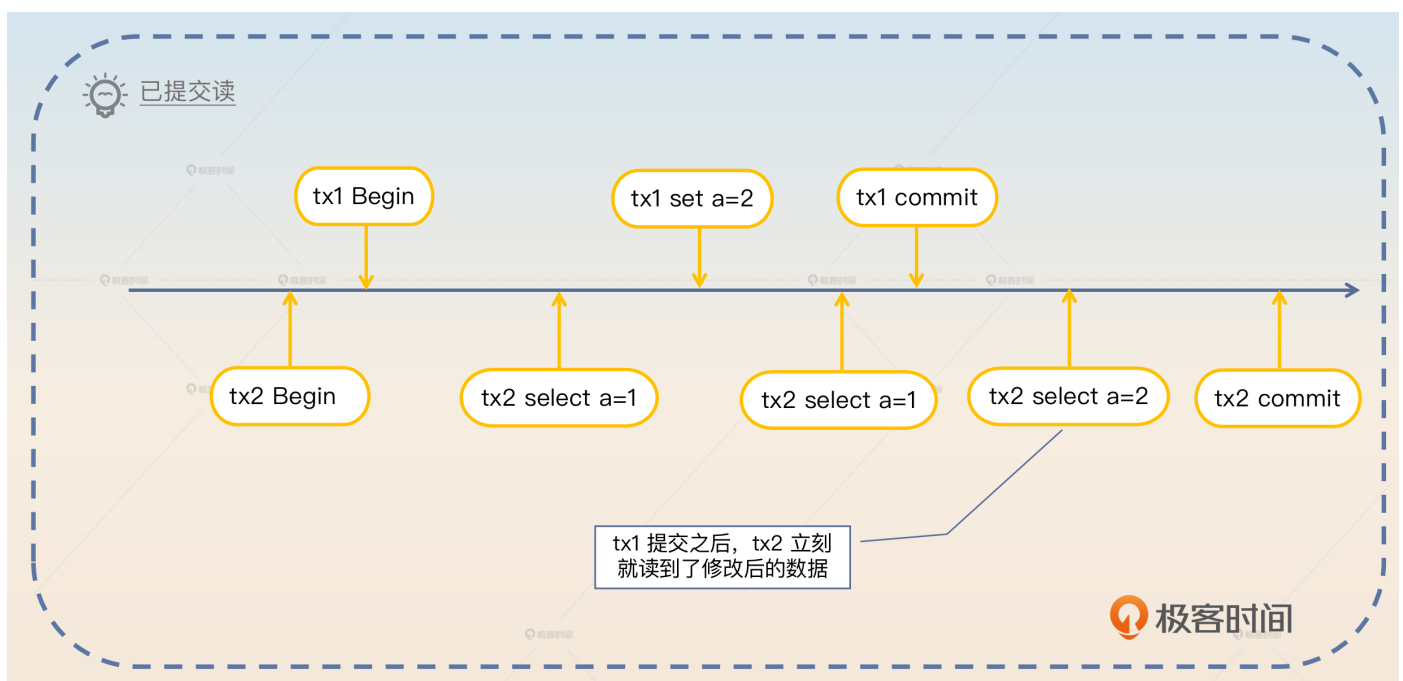
数据库的隔离级别是一组规则，用来控制并发访问数据库时如何分配、保护和共享资源。不同的隔离级别在不同的并发控制策略之间进行调整，从而提供了不同的读写隔离级别和安全性。用人话来说，就是隔离级别代表了一个事务是否了解别的事务以及了解程度怎么样。

MySQL 的隔离级别有四个。

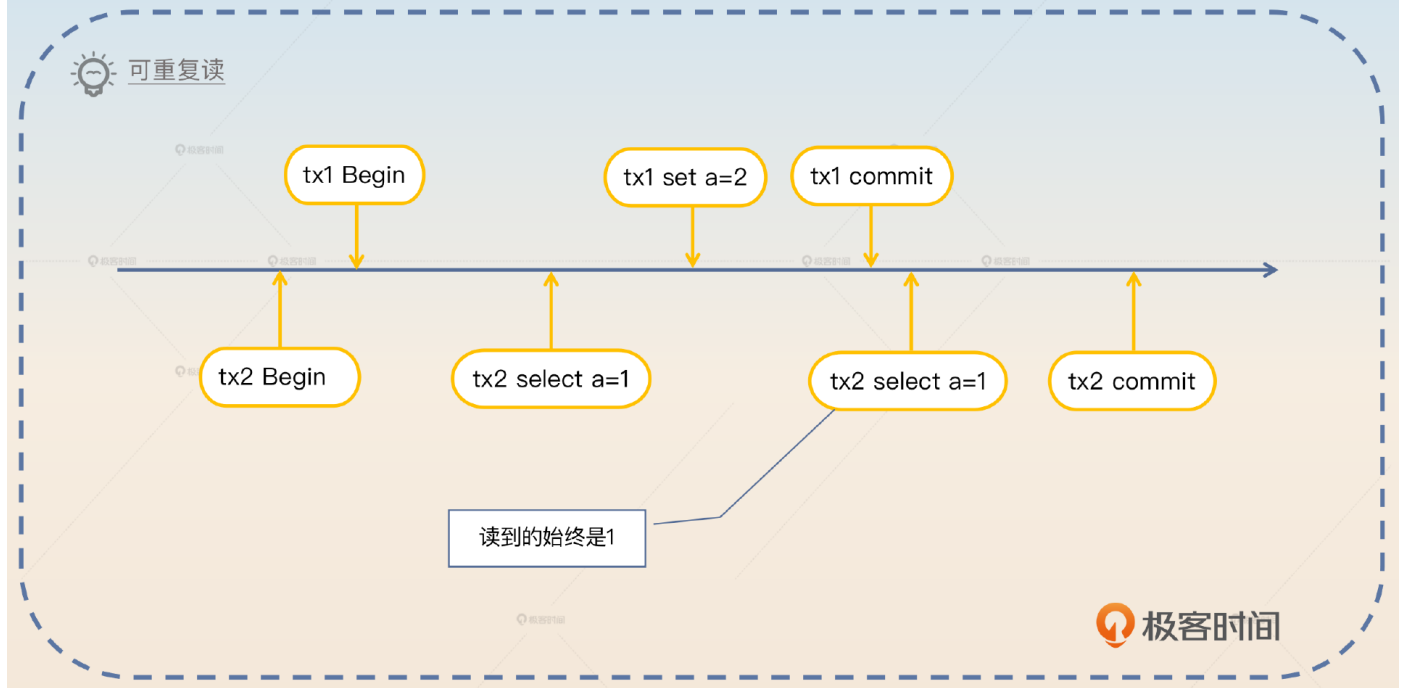
1. **读未提交** (Read Uncommitted) 是指一个事务可以看到另外一个事务尚未提交的修改。



2. **读已提交** (Read Committed, 简写 RC) 是指一个事务只能看到已经提交的事务的修改。这意味着**如果在事务执行过程中有别的事务提交了，那么事务还是能够看到别的事务最新提交的修改。**



3. **可重复读** (Repeatable Read, 简写 RR) 是指在这一个事务内部读同一个数据多次，读到的结果都是同一个。这意味着即便**在事务执行过程中有别的事务提交，这个事务依旧看不到别的事务提交的修改。**这是 MySQL 默认的隔离级别。



4. **串行化** (Serializable) 是指事务对数据的读写都是串行化的。

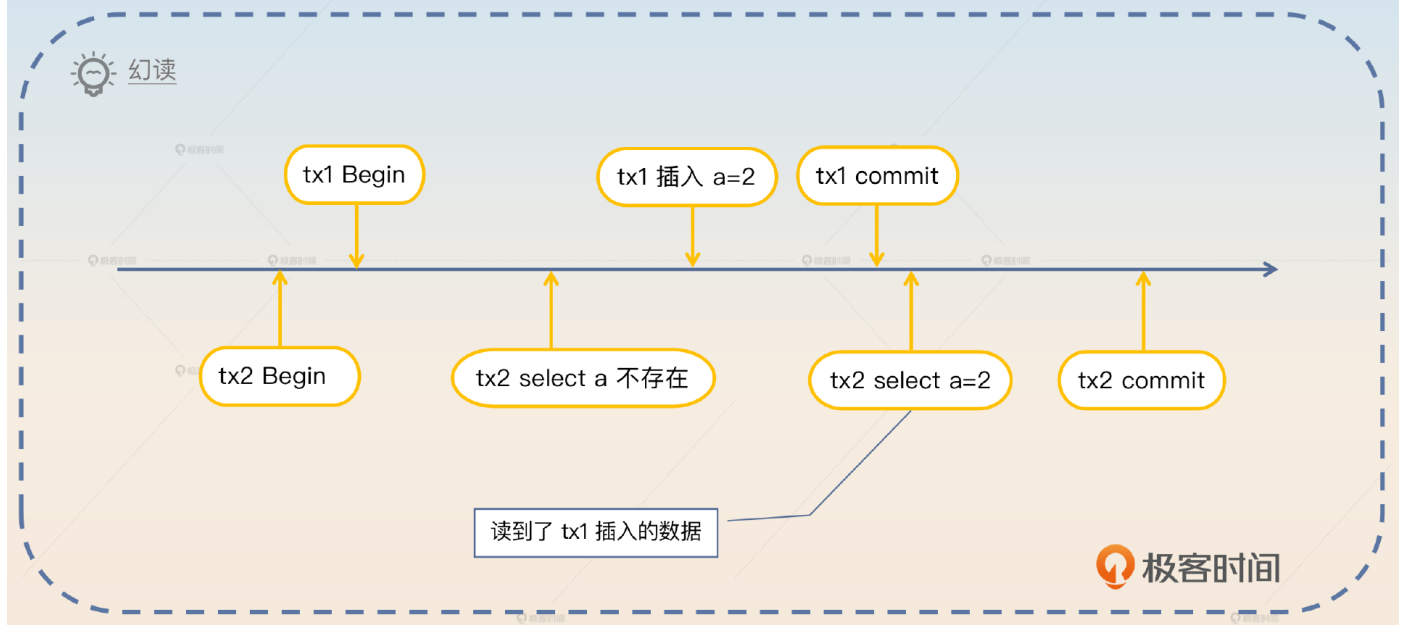
从上到下，隔离性变强但是性能变差了。所以一个提升 MySQL 性能最简单的方式，就是将隔离级别往下调，这也是我们的一个亮点方案。

和隔离级别密切相关的概念是脏读、幻读和不可重复读这三个读异常。

脏读是指读到了别的事务还没有提交的数据。之所以叫做“脏”读，就是因为未提交数据可能会被回滚掉。

不可重复读是指在一个事务执行过程中，对同一行数据读到的结果不同。

幻读是指在事务执行过程中，别的事务插入了新的数据并且提交了，然后事务在后续步骤中读到了这个新的数据。



我们可以用一个表来描述隔离级别和这三种读异常的关系。

隔离级别与读异常	脏读	不可重复读	幻读
读未提交	可能	可能	可能
读已提交	不可能	可能	可能
可重复读	不可能	不可能	不可能（理论上可能）
串行化	不可能	不可能	不可能

极客时间

这里尤其要注意一点，就是理论上来说可重复读是没有解决幻读的。但是 MySQL 因为使用了临键锁，因此它的可重复读隔离级别已经解决了幻读问题。你在面试的过程中不要忘了强调这一点。

此外还有两个相似的概念：快照读和当前读。简单来说，快照读就是在事务开始的时候创建了一个数据的快照，在整个事务过程中都读这个快照；而当前读，则是每次都去读最新数据。MySQL 在可重复读这个隔离级别下，查询的执行效果和快照读非常接近。

版本链

为了实现 MVCC，InnoDB 引擎给每一行都加了两个额外的字段 **trx_id** 和 **roll_ptr**。

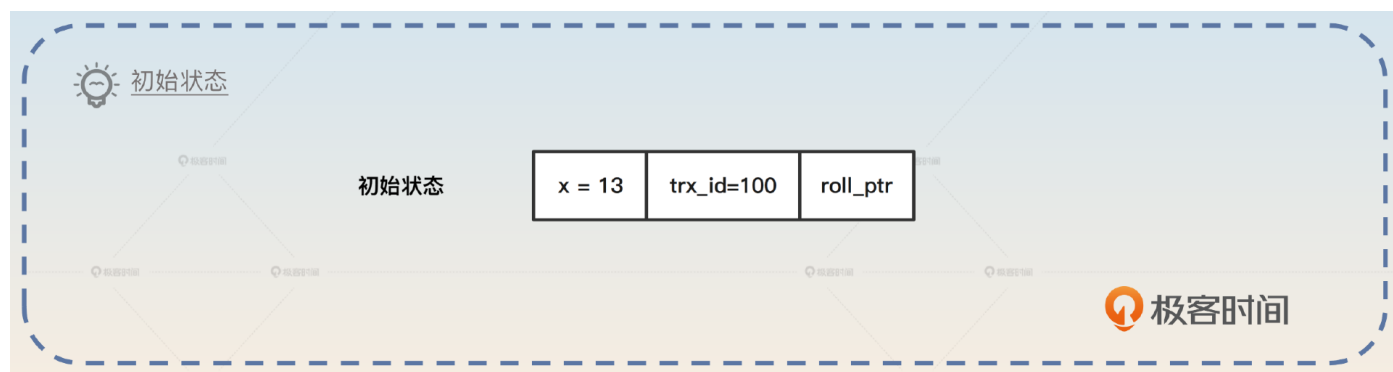
trx_id: 事务ID, 也叫做事务版本号。MVCC 里面的 V 指的就是这个数字。每一个事务在开始的时候就会获得一个 ID, 然后这个事务内操作的行的事务 ID, 都会被修改为这个事务的 ID。

roll_ptr: 回滚指针。InnoDB 通过 roll_ptr 把每一行的历史版本串联在一起。

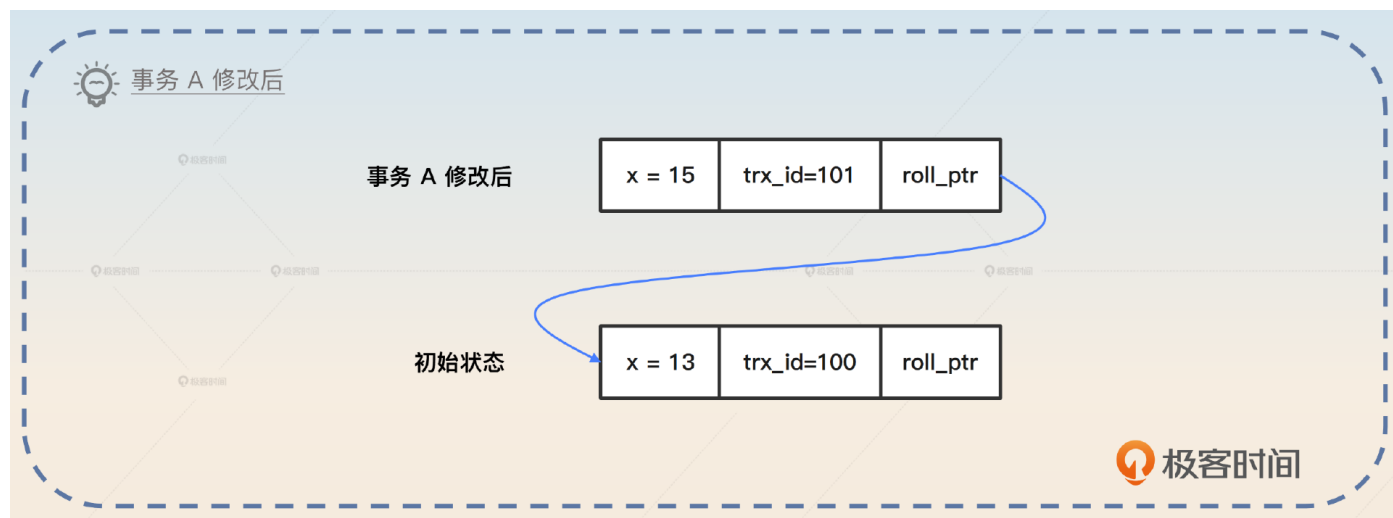
实际上, InnoDB 引擎还隐式地插入了另外一个列 row_id, 如果你没有设置任何主键, 那么这个列就会被当成主键来使用。但是它其实和 MVCC 没太大的关系, 所以你不需要关注。

下面我用一个例子来说明 MVCC 是如何利用这两个列的。

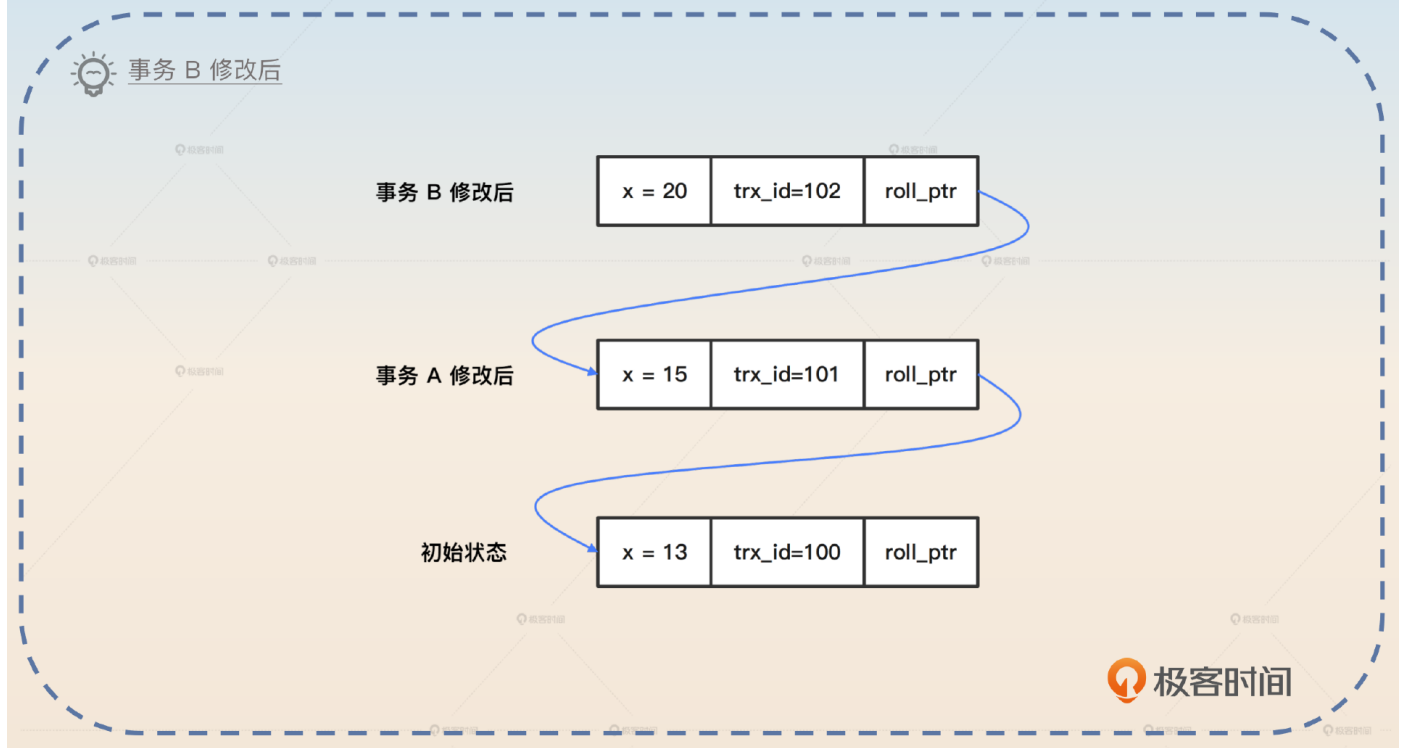
假设最开始我插入了一行数据, 我插入数据的这个事务的 ID 是 100, 那么这个时候数据行看起来是这样的。



假设有一个事务 A 拿到了 ID 101, 然后把 x 的值修改为 15, 那么就会变成这样。



这个时候, 事务 A 修改后的 roll_ptr 会指向初始状态的数据。假如现在再来一个事务 B 拿到 ID 102, 要把数据 x 修改成 20, 那么就会变成下面这样。



这条链就是大名鼎鼎的版本链。这个版本链存储在所谓的 undolog 里面，undolog 我们下一节课会详细讨论。

现在问题来了，假如这个时候我有一个新的事务 C，我要读 x 的值，那么我该读取 trx_id 为几的数据呢？这就涉及到了另外一个和 MVCC 紧密相关的概念：Read View。

Read View

Read View 你可以理解成是一种可见性规则。前面你已经知道了 undolog 里面存放着历史版本的数据，当事务内部要读取数据的时候，Read View 就被用来控制这个事务应该读取哪个版本的数据。

Read View 最关键的字段叫做 m_ids，它代表的是当前已经开始，但是还没有结束的事务的 ID，也叫做活跃事务 ID。

Read View **只用于已提交读和可重复读**两个隔离级别，它用于这两个隔离级别的不同点就在于**什么时候生成 Read View**。

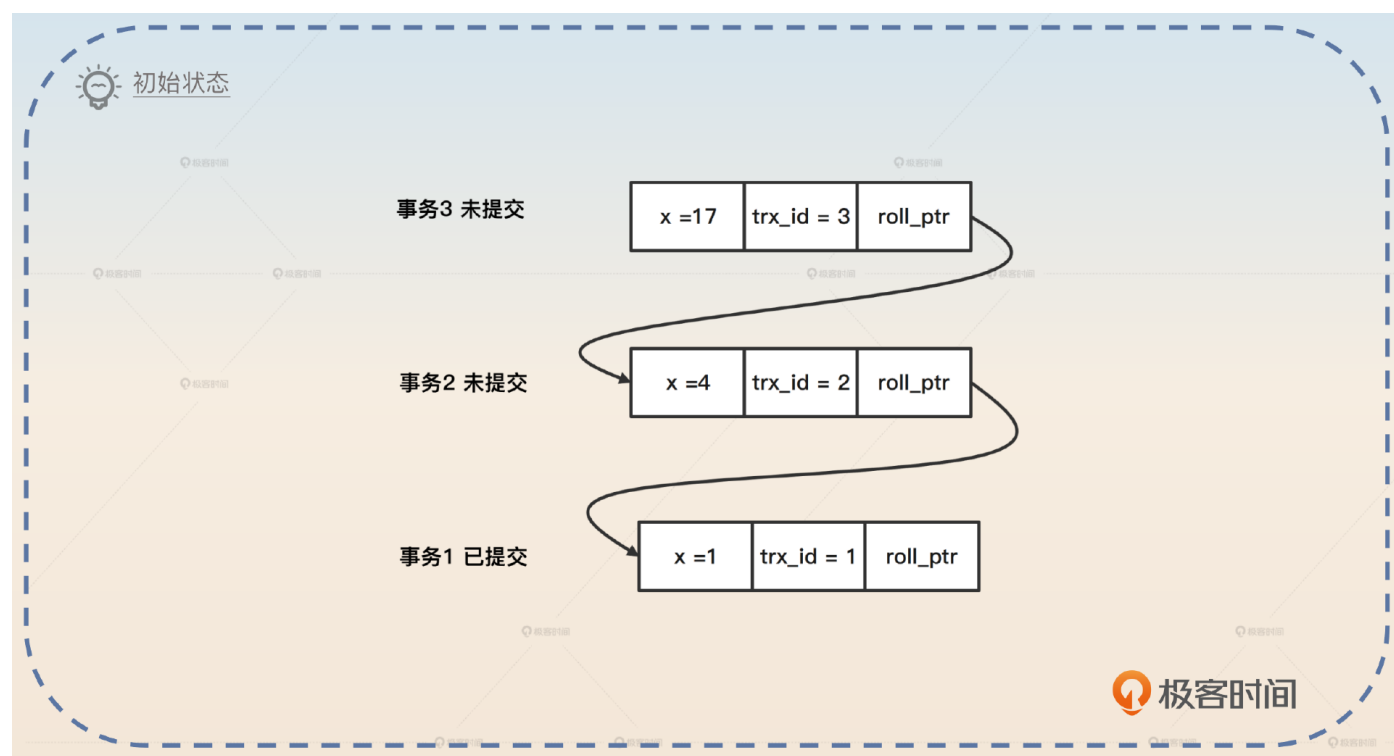
已提交读：事务每次发起查询的时候，都会重新创建一个新的 Read View。

可重复读：事务开始的时候，创建出 Read View。

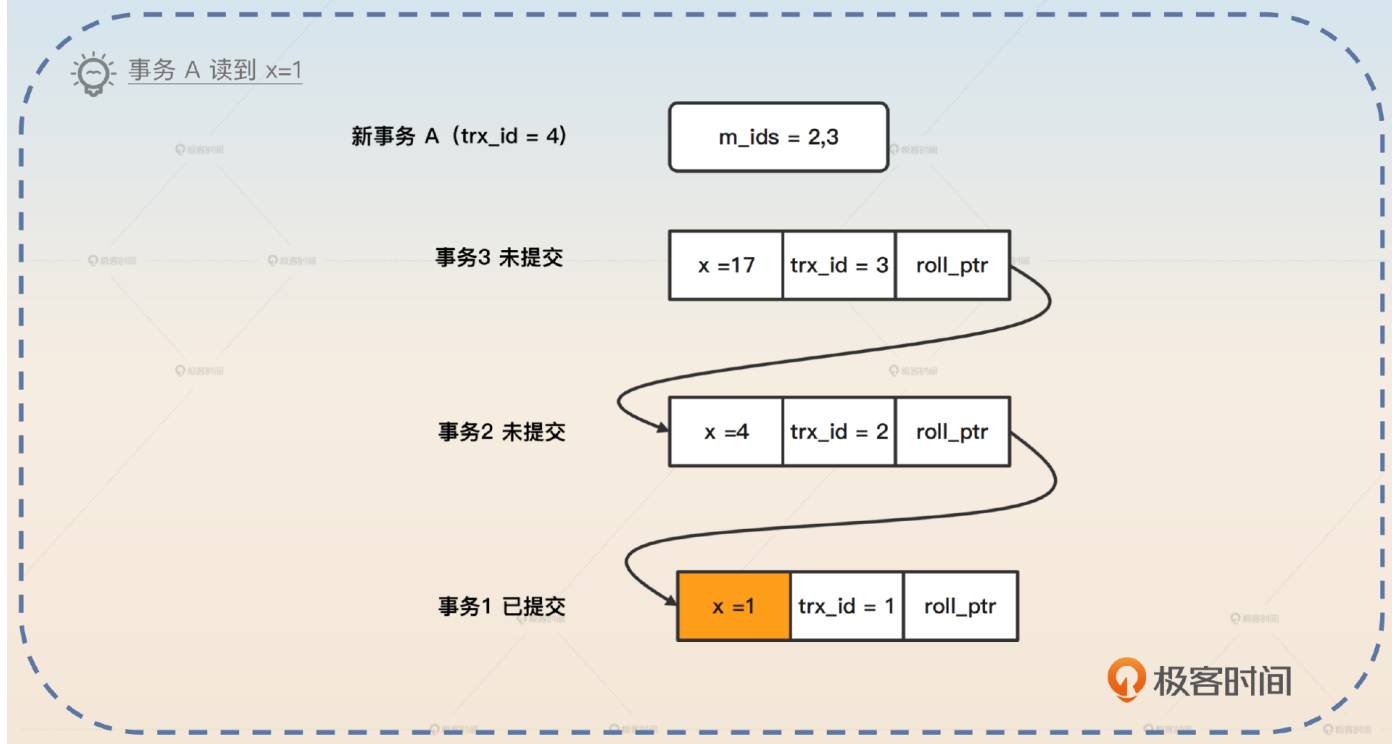
已提交读就像你的渣男朋友，你每次见到他，他都会换一个新对象；而可重复读就是一个痴情男，你每次见到他，看到的都是他高中时候谈的对象。

Read View 与已提交读

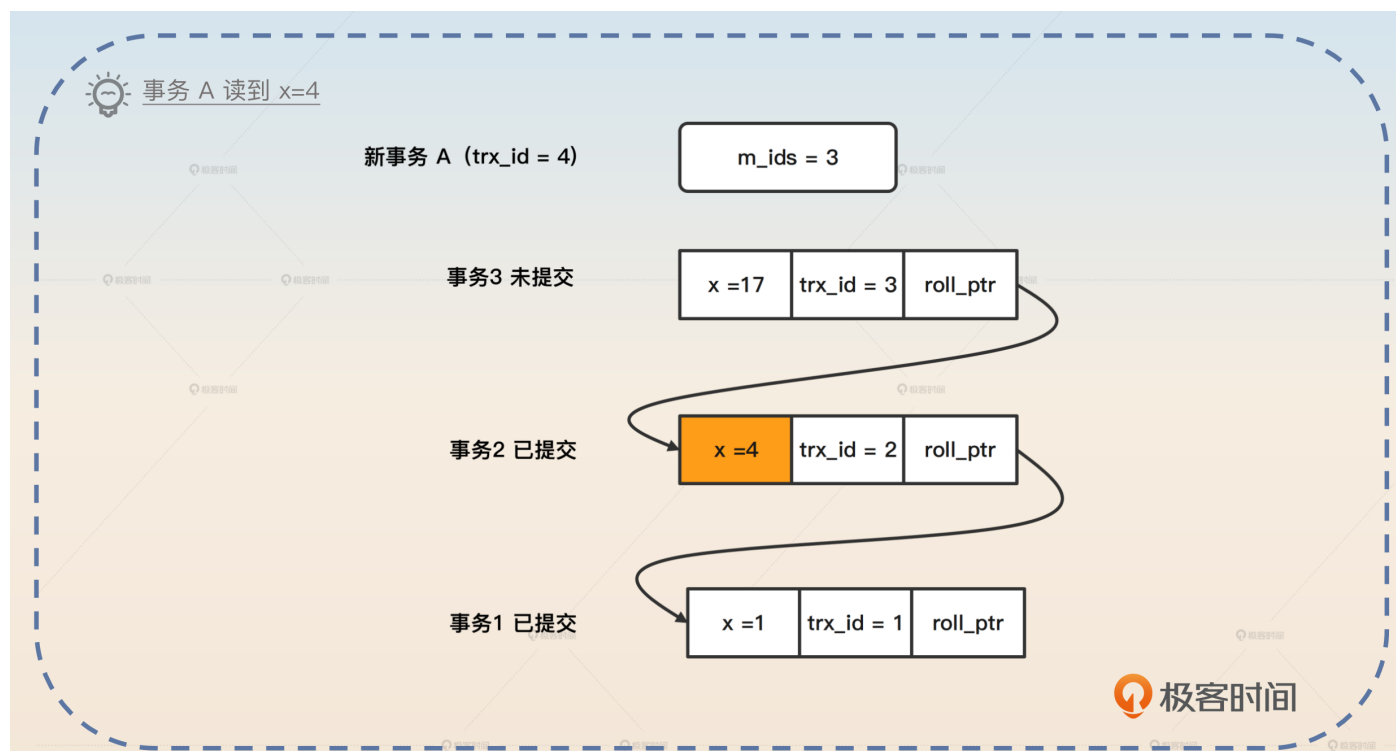
在已提交读的隔离级别下，每一次查询语句都会重新生成一个 Read View。这意味着在事务执行过程中，Read View 是在不断变动的。现在来看一个例子，假如说现在已经有三个事务了，状态分别是已提交、未提交、未提交。



假如说现在新开了一个事务 A，分配给它的 ID 是 4。如果这个时候 A 开始查询 x 的值，那么 MySQL 会创建一个新的 Read View，其中 $m_ids = 2, 3$ 。事务 A 发现最后一个已经提交的是事务 $trx_id = 1$ ，对应的 x 的值是 1。于是事务 A 读到 $x = 1$ 。



如果这个时候事务 2 提交了，事务 A 再次读取 x ，这个时候 MySQL 又会生成一个新的 **Read View** $m_ids=3$ 。因此事务 A 会读取到 $x = 4$ 。



Read View 与可重复读

在可重复读的隔离级别下，数据库会在事务开始的时候生成一个 Read View。这意味着整个 Read View 在事务执行过程中都是稳定不变的。我们用前面的例子来说明，就是在事务 A 开始的时候就会创建出来一个 Read View $m_ids=2,3$ 。



事务 A 开始就创建出来 Read View

新事务 A (trx_id = 4)

m_ids = 2,3

事务3 未提交

x = 17

trx_id = 3

roll_ptr

事务2 未提交

x = 4

trx_id = 2

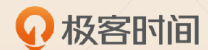
roll_ptr

事务1 已提交

x = 1

trx_id = 1

roll_ptr



如果这时候事务 A 去读 x 的数据，毫无疑问，读出来的是 x=1。



事务 A 读到 x=1

新事务 A (trx_id = 4)

m_ids = 2,3

事务3 未提交

x = 17

trx_id = 3

roll_ptr

事务2 未提交

x = 4

trx_id = 2

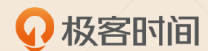
roll_ptr

事务1 已提交

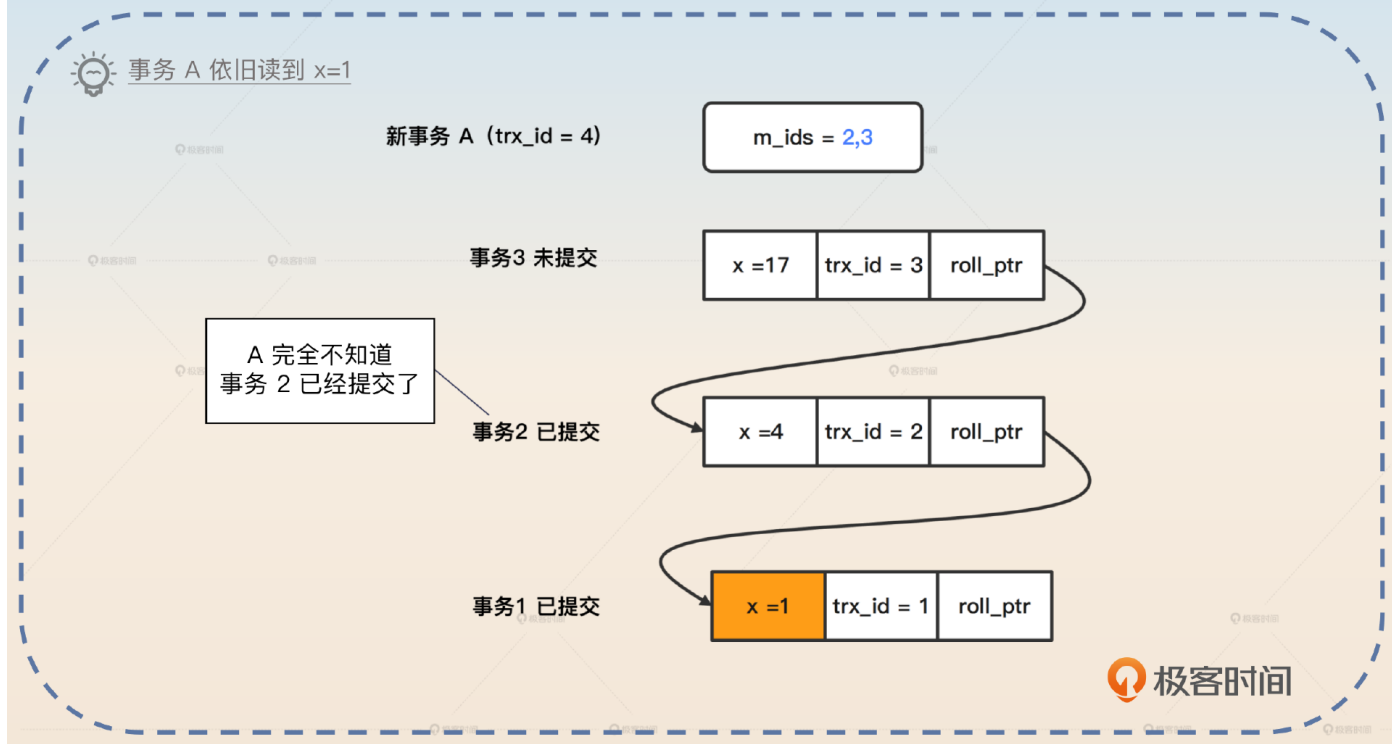
x = 1

trx_id = 1

roll_ptr



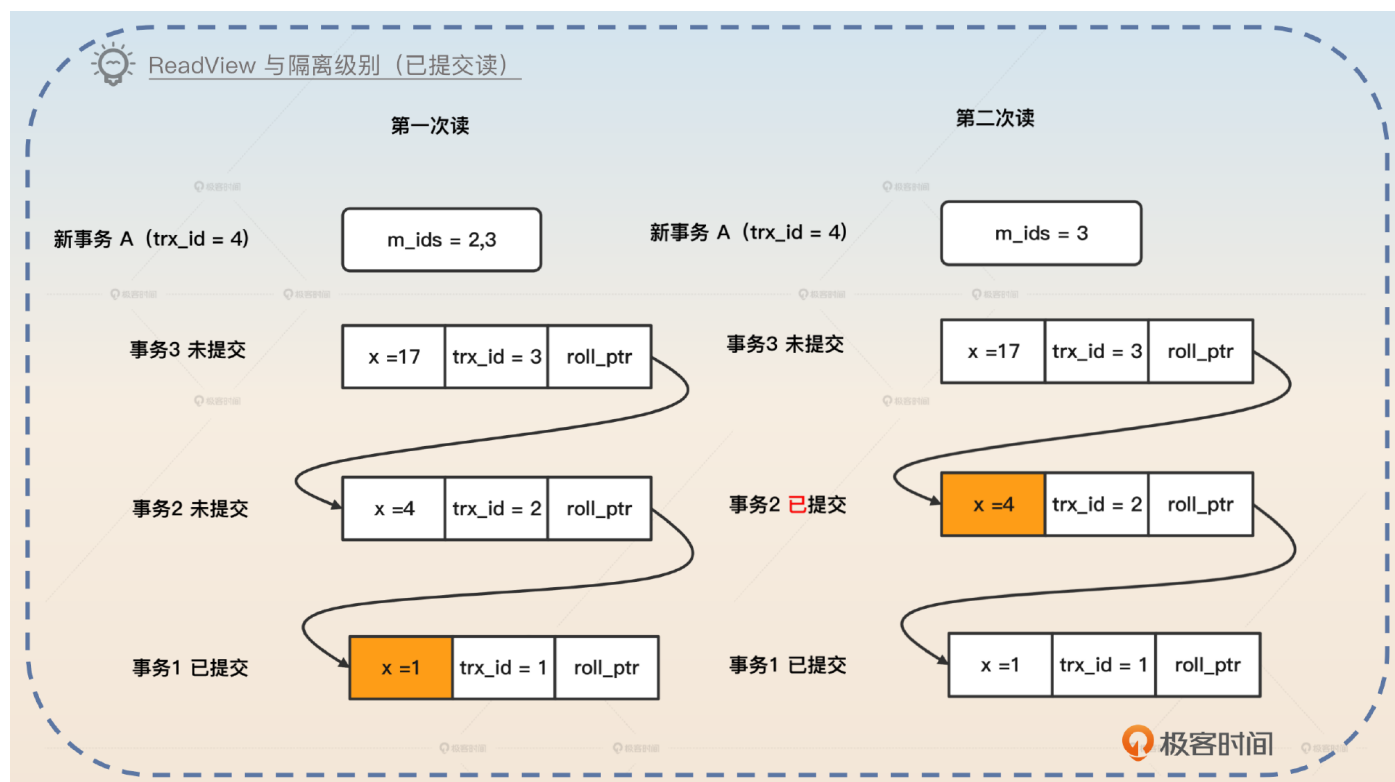
如果这时候事务 2 提交了，然后事务 A 想要再去读 x 的值，Read View 不会发生变化，还是 m_ids = 2,3。所以你可以看到，虽然事务 2 提交了，但是事务 A 完全不知道这回事，因此它还是读到 x=1。



万一这时候有一个新事务 ID = 5 开始了，并且也提交了。那么事务 A 并不会读取这个新事务的数据，因为新事务 ID 已经大于事务 A 的 ID 了 ($5 > 4$)，事务 A 知道这是一个比它还要晚的事务，所以会忽略新的事务的修改。

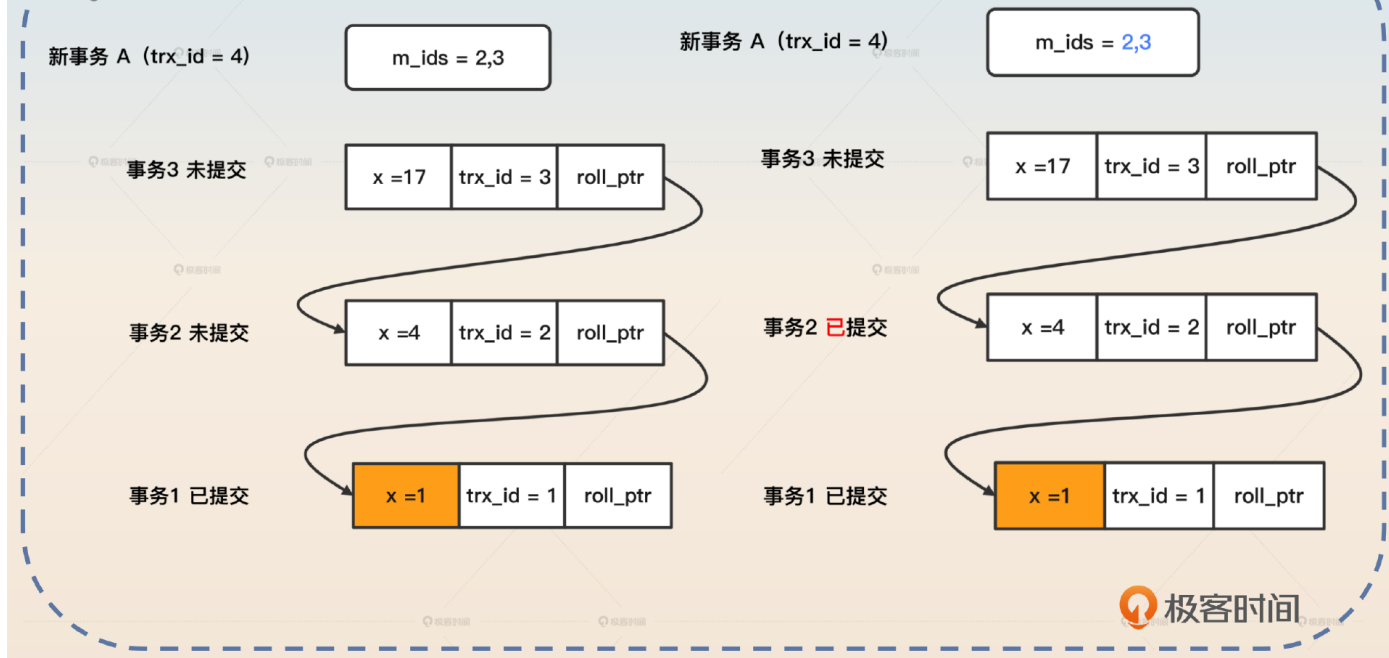
Read View 小结

我把前面的内容整合在一起，画成了图，你可以参考。





ReadView 与隔离级别（可重复读）



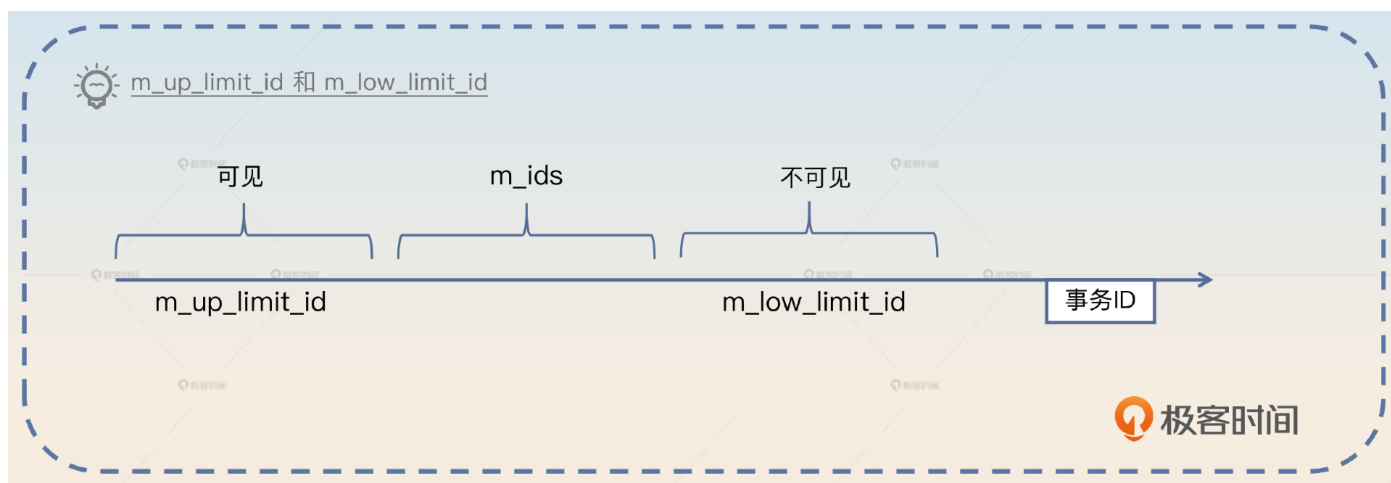
这里我只提到了 `m_ids`，实际上和 Read View 相关的概念还有三个。

`m_up_limit_id`是指 `m_ids` 中的最小值。

`m_low_limit_id`是指下一个分配的事务 ID。

`m_creator_trx_id`当前事务 ID。

那么可见性如下图所示：



`m_up_limit_id` 在左边，而 `m_low_limit_id` 在右边，你不要记错了。实际上，`m_up_limit_id` 和 `m_low_limit_id` 你记不住也没关系，它不影响你对 MVCC 和 ReadView 核心逻辑的理解。

面试准备

有了前面基础知识的铺垫，现在你应该可以分清重点了。那么在面试中我们应该关注哪些细节呢？

首先你需要了解清楚你们公司数据库的隔离级别，如果你们公司设置的不是默认的隔离级别，那么你就要搞清楚为什么不使用默认隔离级别。尤其是用了未提交读、串行化两个隔离级别，更加要弄清楚为什么决定使用这两个隔离级别。

在面试过程中，面试官会出一些让人很难反应过来的问题。比如说面试官会口头构造一条版本链。

我现在有三个事务，ID 分别是 101、102、103。如果事务 101 已经提交了，但是 102、103 还没提交。这个时候，我开启了一个事务，准备读取数据，那么我读到的是哪个事务的数据？

如果这时候你回答读取到事务 101 的数据，那么面试官就进一步追问。

如果这时候事务 103 提交了，但是 102 还没提交，那么会读到谁的呢？

你就要回答根据隔离级别来了。

这种问题很难反应过来是因为你需要快速在脑海里面建立起整个版本链，然后综合考虑隔离级别以及谁先提交谁后提交，才能正确回答出来。你想一下你刚才读到那一段话的时候，是不是有一种迷糊的感觉？在面试那种紧张的氛围下，你只会觉得迷糊，难以反应过来。

我的建议是你只要对这一类问题有一个心理预期就可以了。在面试的时候要是一时半会没办法回答出来，可以请求面试官再说一遍，并且说慢一点。如果面试官好说话，你还可以借助纸笔，直接将这个东西画出来，再来分析最终读到的是什么数据。

基本思路

有些时候面试官会在面了锁之后，将话题引到 MVCC，问你为什么有了锁，还需要 MVCC？你在回答的时候要答出关键词**避免读写阻塞**。

单纯使用锁的时候，并发性能会比较差。即便是在读写锁这种机制下，读和写依旧是互斥的。而数据库是一个性能非常关键的中间件，如果某个线程修改某条数据就让其他线程都不能读这条数据，这种性能损耗是无法接受的。所以 InnoDB 引擎引入了 MVCC，就是为了减少读写阻塞。

大部分时候，面试官在问 MVCC 的时候，都是直接问你这几个问题。

你是否了解 MVCC？

MVCC 是什么？

MySQL 的 InnoDB 引擎是怎么控制数据并发访问的？

当一个线程在修改数据的时候，另外一个线程还能不能读到数据？

这时候你就要简明扼要地把原理解释清楚。按照**基本定义、实现机制、隔离级别**的逻辑顺序来回答。

MVCC 是 MySQL InnoDB 引擎用于控制数据并发访问的协议。MVCC 主要是借助于版本链来实现的。在 InnoDB 引擎里面，每一行都有两个额外的列，一个是 `trx_id`，代表的是修改这一行数据的事务 ID。另外一个 `roll_ptr`，代表的是回滚指针。InnoDB 引擎通过回滚指针，将数据的不同版本串联在一起，也就是版本链。这些串联起来的历史版本，被放到了 `undolog` 里面。当某一个事务发起查询的时候，MVCC 会根据事务的隔离级别来生成不同的 Read View，从而控制事务查询最终得到的结果。

这里的话术非常简洁，基本上没有涉及任何细节，但是又提及了足够多的关键词。

首先，在回答里我们提到了 `undolog`，那么接着面试官就可能追问 `undolog`、`redolog` 或者 `binlog` 的细节，这一部分可以把话题引到下一节课的内容。

其次，在回答中我们提到了隔离级别，并且提到了 Read View 是和隔离级别有关的东西，那么面试官就会非常深入地问隔离级别的基本定义，MVCC 是怎么利用 Read View 来实现已提交读和可重复读的。

在回答的时候，要先解释清楚**四个隔离级别和三个读异常**，然后强调一下 InnoDB 引擎。

在 MySQL 的 InnoDB 引擎里面，使用了临键锁来解决幻读的问题，所以实际上 MySQL InnoDB 引擎的可重复读隔离级别也没有幻读的问题。一般来说，隔离级别越高，性能越差。所以我之前在公司做的一个很重要的事情，就是推动隔离级别降低为已提交读。

这个回答的最后，你就可以尝试将话题引导到下面的亮点方案中。

亮点方案

这一个亮点方案重点在于描述清楚两方面的内容。

1. 推动公司将隔离级别从默认的可重复读降低为已提交读。
2. 在已提交读的基础上，万一需要利用可重复读的特性，该怎么办？

从前面的内容中你已经知道，MySQL 的默认隔离级别是可重复读，实际上互联网的很多应用都调整过这个隔离级别，降低为已提交读。

那么你在面试的时候可以考虑使用这个来作为你的亮点方案。首先你要强调**为什么要改**。

最开始我来到公司的时候，我们的数据库隔离级别都是使用默认的隔离级别，也就是可重复读。但其实我们的业务场景很少利用可重复读的特性，比如说几乎全部事务内部对某一个数据都是只读一次的。

并且，可重复读比已提交读更加容易引起死锁的问题，比如说我们之前就出现过一个因为临键锁引发的死锁问题。而且已提交读的性能要比可重复读更好。所以综合之下，我就推动公司去调整隔离级别，将数据库的默认隔离级别降低为已提交读。

在这种情况下，面试官可能会追问你：“在调整了事务级别之后，万一需要可重复读的特性了，你怎么办？”

首先你要理解在什么样的场景下你才会需要可重复读这个隔离级别。

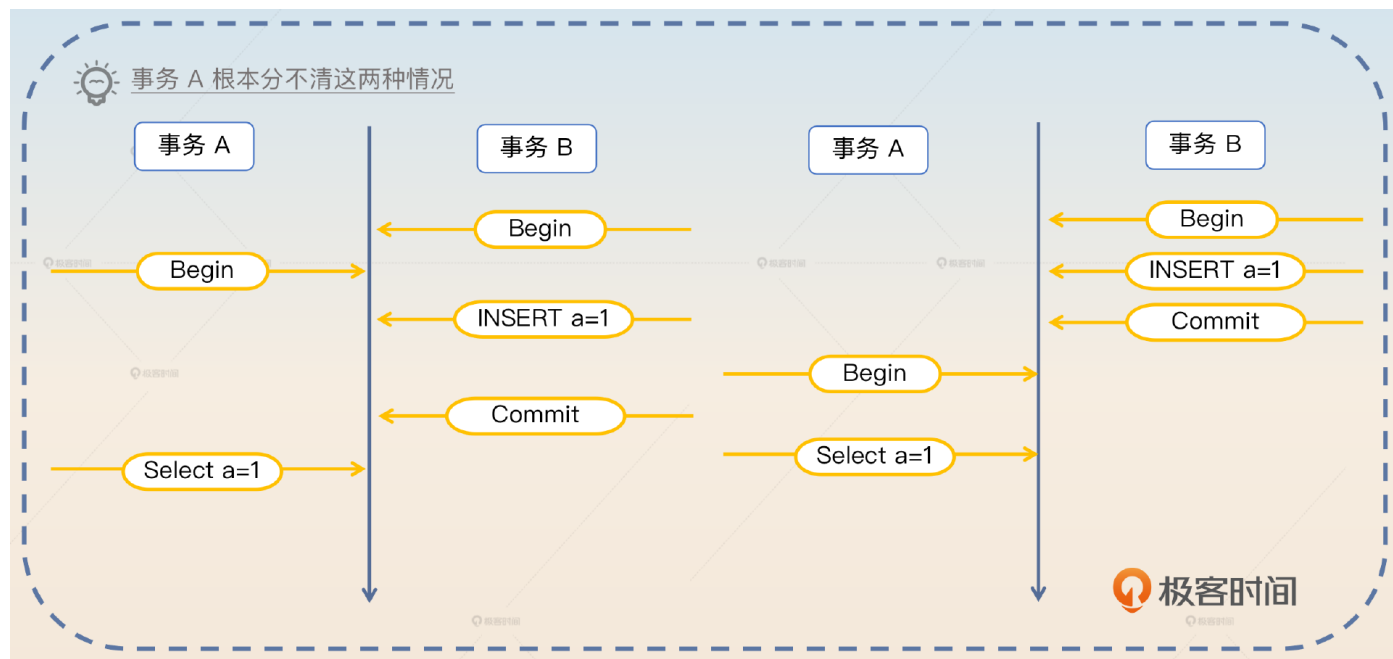
你需要在事务中发起两次同样的查询，并且你希望两次得到的结果是一样的。

你需要避开幻读，也就是事务开始之后，即便有别的事务插入了数据并且提交了，你也不希望读到这个新数据。

但是仔细想想，你真的存在这种场景吗？或者说，你真的没得选，以至于一定要使用可重复读这个隔离级别吗？

答案是几乎没有。大部分出现可重复读的需求都是因为代码没有写好，或者说至少可以通过改造业务来实现。比如说常见的可重复读，既然你需要读多次，那么自然可以在第一次读完之后缓存起来。

不过幻读是没有办法通过业务改造来解决的。但是在业务层面上，幻读一般不会被认为是一个问题，原因有两点：一是你分不清是不是幻读。比如说你在事务 A 里面读到了一条数据，你判断不出来它是在事务 A 开始之前就插入的，还是在事务 A 开始之后，事务 B 才插入并且提交的。



二是事务提交往往意味着业务已经结束，所以读到一个已经提交的事务的数据，不会损害业务的正确性。也就是说，如果事务 A 在开始之后，事务 B 才插入数据并且提交。那么这个时候事务 A 完全可以认为事务 B 所在的整个业务已经结束了，那么读出来也没什么问题。

所以你可以这么回答，关键词是**改造业务**。

正常来说我是不推荐使用可重复读的，因为在我们的业务环境下想不到有什么场景非得使用可重复读这个隔离级别。

之前在推动降低隔离级别的时候，我其实重构过一些业务。这一类业务就是在一个事务里面发起了两个同样的查询，比如说在 UPDATE 之后又立刻查询，这种查询还必须走主库，不然会有主从延迟的问题。

这种业务可以通过缓存第一次查询的数据来避免第二次查询。但是这种改造一般是避不开幻读的。不过在业务上幻读一般不是问题。一方面是业务层面上区分不出来是否是幻读。另外一方面，事务提交了往往代表业务已经结束，那么发生幻读了，业务依旧是正常的。比如说事务 A 读到了事务 B 新插入的数据，但是事务 B 本身已经提交了，那么事务 A 就认为事务 B 所在的业务已经完结了，那么读到了就读到了，并不会出什么问题。

但是这种回答，如果遇上较真的面试官，他依旧会觉得不满意，那么你就可以使出最后的兜底手段，关键词是**指定隔离级别**。

万一不能改造业务，那么还有一个方法，就是直接在创建事务的时候指定隔离级别。我前面调整的都是数据库的默认隔离级别，实际上还可以在 Session 或者事务这两个维度上指定隔离级别。

如果你记不住或者难以理解如何改造业务代码，你只需要回答这一点就可以。

面试思路总结

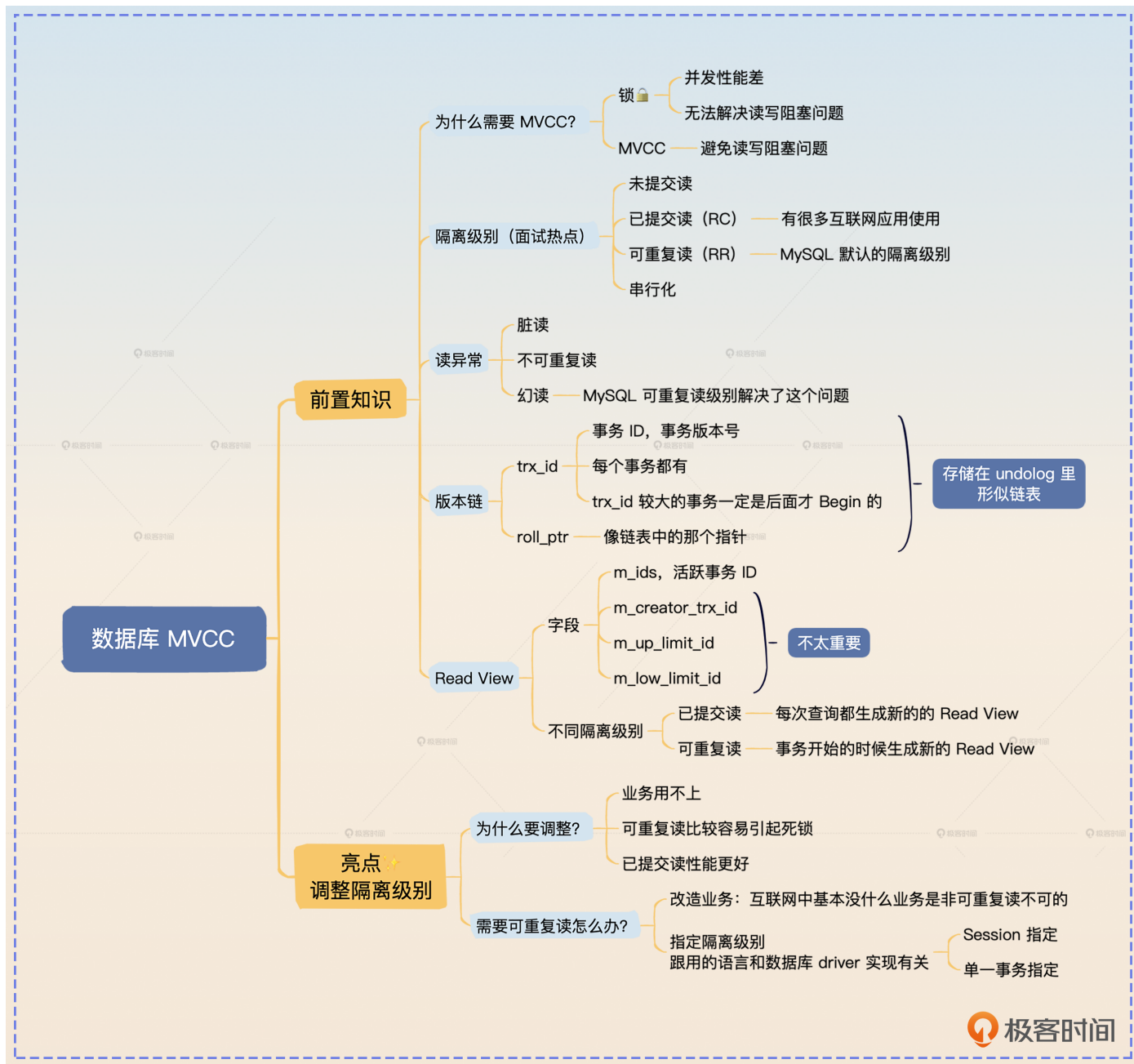
今天我们重点学习了 MVCC 的基本原理，这里我总结一下这节课的主要内容。

1. 你需要记住为什么需要 MVCC，尤其是在有锁机制的情况下，为什么还需要 MVCC？主要是为了读写并发。
2. 你需要记住四个隔离级别：未提交读、已提交读、可重复读、串行化，以及和隔离级别密切相关的三个读异常：脏读、不可重复读、幻读。
3. 你需要记住 MVCC 是如何构造版本链的。
4. 你需要记住 Read View 在不同的隔离级别下是如何运作的。

最后我也给出一个调整隔离级别的亮点方案，面试的时候要抓住两个关键点。

为什么要调整为已提交读？主要原因有两个：一是因为业务用不上，二是为了提升性能。

在调整之后真的需要可重复读隔离级别该怎么办？你就按照我给出的话术回答就好了。



思考题

最后你来思考几个问题。

MVCC 只作用于已提交读和可重复读，那么 InnoDB 是怎么处理其他两个隔离级别的？

你有没有遇到过看起来真的要可重复读这个隔离级别的问题？如果遇到了，你现在有办法在已提交读这个隔离级别上解决这个问题吗？

欢迎你把你的答案分享在评论区，也欢迎你把这节课的内容分享给需要的朋友，我们下节课再见！

到了后面的几面，面试官很有可能会问你这种影响很大的事情你是怎么推进的？你是如何说服同事的？你是如何说服上司的？这一类问题就属于软技能。类似这种推进某件事情的策略，你基本上都可以按照准备充分、公开决议、小步推进、全面铺开四步来回答。

准备充分：当你准备推进一件事的时候，要充分调研实际情况，提出针对性的方案。需要强调你知道为什么不做不行，以及如果要做该怎么做。就拿这节课的内容来说，你已经调研清楚了业务上确实用不到可重复读这个隔离级别，并且公司也的确出过死锁的案例。而你的计划是先从比较简单的不重要的数据库开始，降低隔离级别，经过验证之后再推广。或者采用更简单的方案，就是老的数据库不变，新的数据库就使用已提交读来作为默认的隔离级别。

公开决议：正常来说，一般要先和关键人物沟通，取得支持。然后在公开会议上抛出议题，取得大多数人的支持。这一步其实有点自保的意味，因为群体决策就意味着群体负责。并且取得大多数人的支持之后，推进一件事会更加容易。

小步推进：放到这节课就是按照你的计划，先改造不重要业务的数据库，或者只在新的数据库上应用。也就是说在实施的初期，先小规模推进。这样可以验证方案的正确性，也可以在出事的时候将影响范围控制住。

全面铺开：你有了成熟的改造经验之后，就可以制定操作规范之类的东西了，让业务的负责人自己选择合适的时机进行切换。在经过了前一个步骤的验证之后，你对方案的弊病、落地可能出现的问题就都心中有数了。这时候就可以全面铺开了。

这节课你第一次接触到了软技能面试相关的内容。软技能面试在整个面试过程中，有一点成事不足败事有余的味道。意思是说，如果你硬实力——技术实力不达标，软技能面出花来，也没用，这就是成事不足；但是如果你硬实力很强，但是后面老大和部门负责人面试你的时候觉得你软技能不行，那你依旧会被淘汰，这就是败事有余。

软技能面试其实也是需要准备的，你平时准备面试不要忽略这一点。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 12 | 数据库锁：明明有行锁，怎么突然就加了表锁？

下一篇 14 | 数据库事务：事务提交了，你的数据就一定不会丢吗？

